

Introduction to Operating Systems

Ivy Peng
Associate Professor
EECS, KTH

Outline

- **What is an OS**
- A brief history (video lecture)
- OS organization
- OS structure

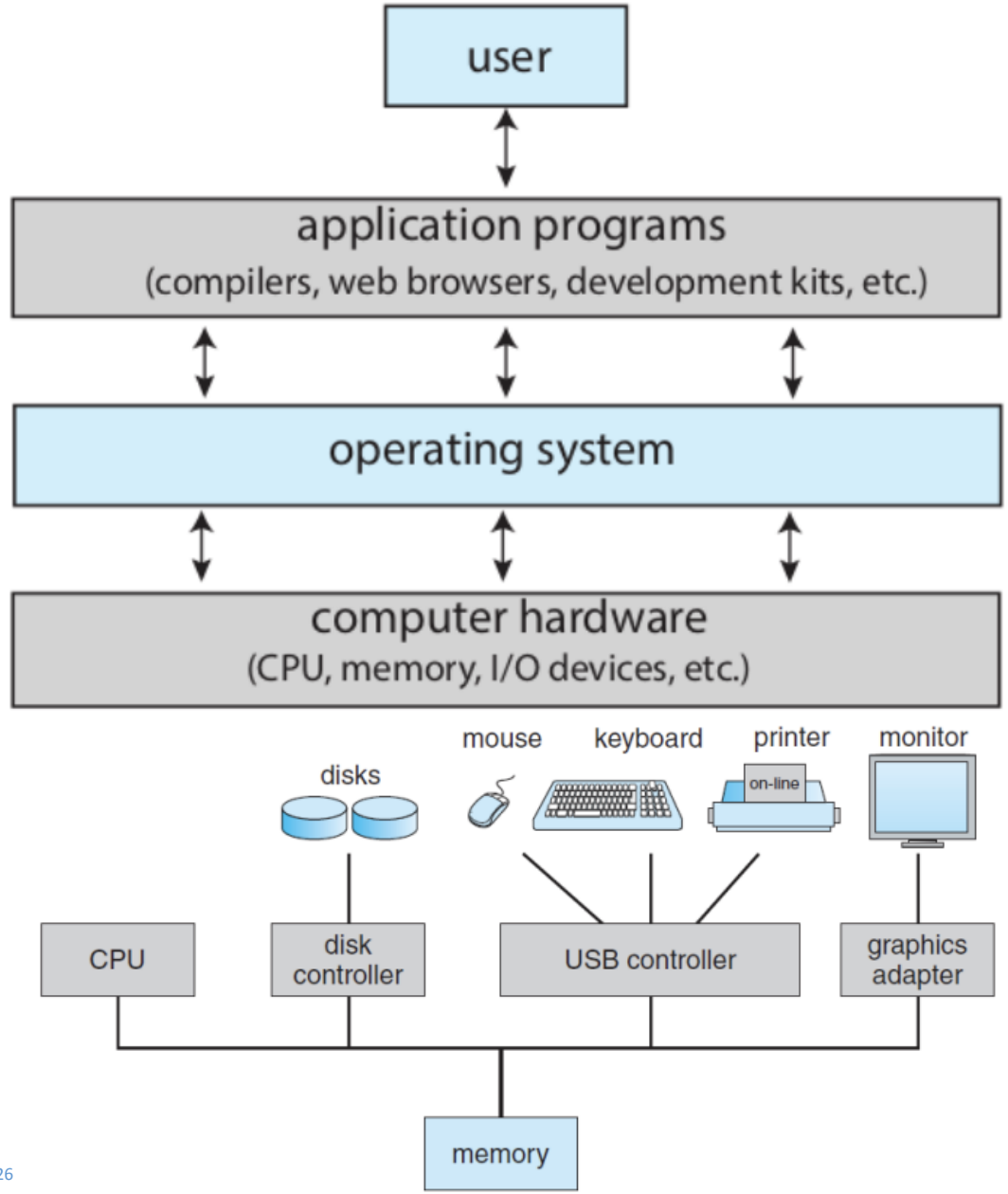
Lecture 1: Introduction to OS

- 0.1.1 Lecture: What is an Operating System? - Video Overview
- 0.1.2 Lecture: A Brief History of Operating Systems - Video Overview
- 0.1.3 Video: Recap of Introduction to OS

What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware.

- An overview video is available on Canvas



Goals of an Operating System

01

Execute users' programs on one system with isolation.

02

Make the computer system convenient to use.

03

Use the computer hardware in an efficient manner.

What do Operating Systems do?

- OS is a resource allocator
  - Manages all resources: CPU, memory, disk, network, …
  - Decides between conflicting requests for efficient and fair resource use.
- OS is a controller
  - Controls execution of programs to prevent errors and improper use of the computer.



The OS and the users share the hardware and software resources of the computer system.

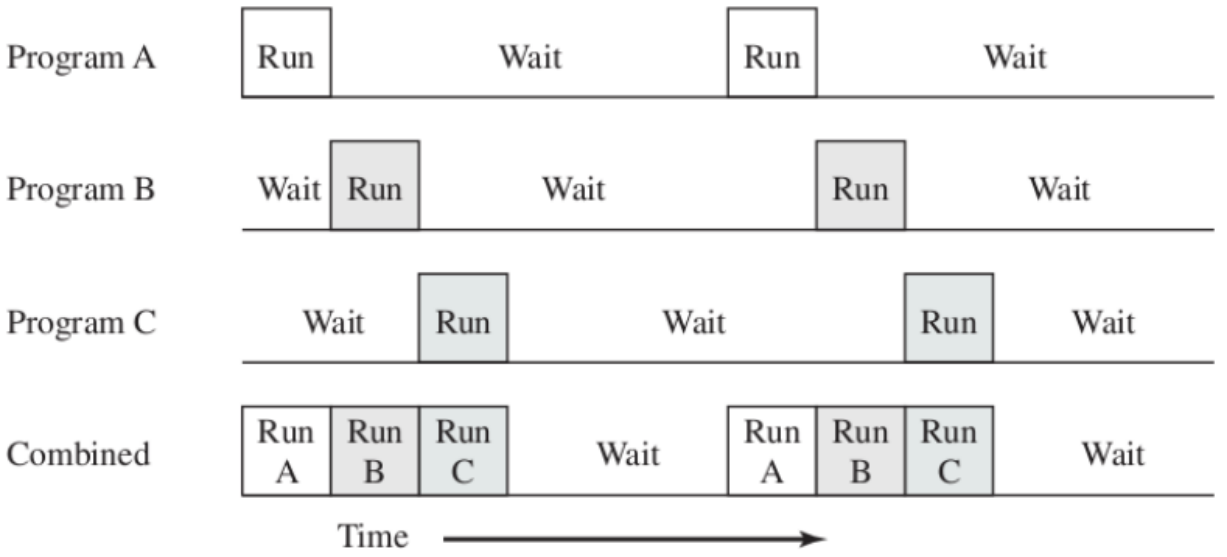


We need to ensure that an error in a user program affects only the one program running.

E.g., getting stuck in an infinite loop

Multiprogramming

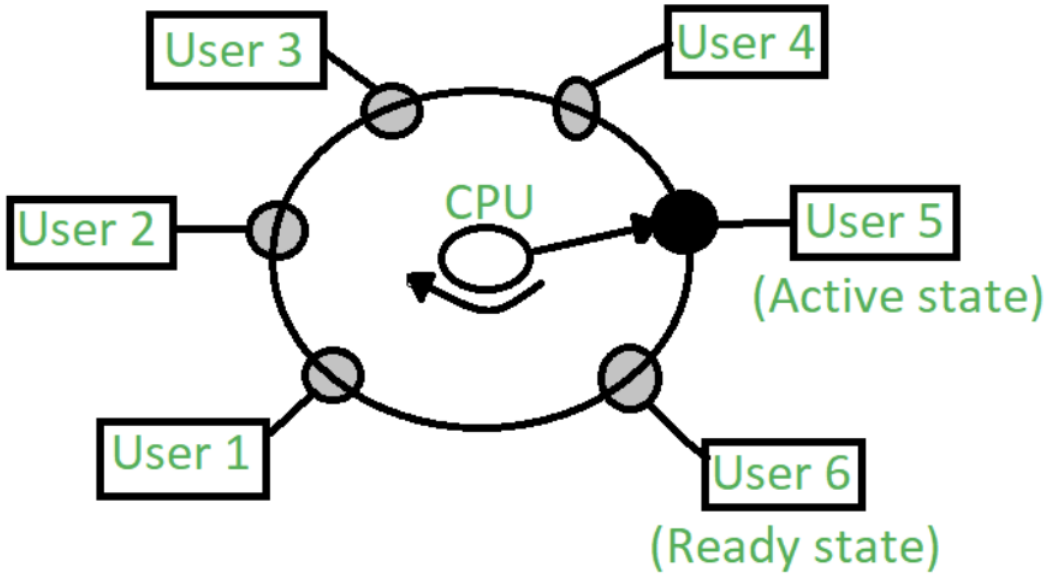
- OS organizes jobs (processes) so the CPU always has one job to execute.
- Multiprogramming is needed for efficiency.
  - Job scheduling decides which job is selected to run, i.e., use hardware resources like CPU
  - When a job waits (for I/O operations, e.g., read from disk), OS switches to run another job.



Courtesy: W. Stallings, Operating Systems: Internals and Design Principles, 2011

Multitasking / Timeslicing

- The CPU switches among different jobs so frequently that users can interact with each job while running, creating **interactive computing**.
- Provide each user with a small time-share of the computer.
- Each job typically executes for only a short time.
- If several jobs are ready to run at the same time, need **CPU scheduling** to select a job to run



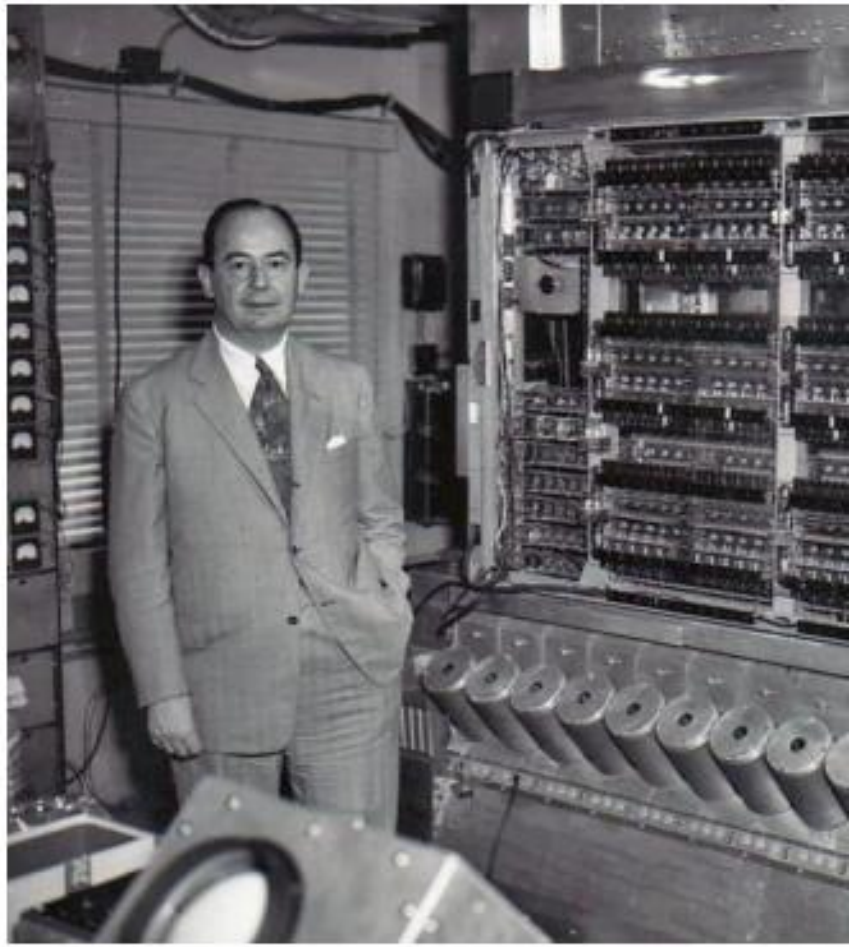
Courtesy: https://www.geeksforgeeks.org/time-sharing-operating-system/

Outline

- What is an OS
- **A brief history (video lecture)**
- OS organization
- OS structure

1st Generation: 1945-1955

- No operating system
- Human was the operator and programmer.
- Computers were programmed by physically **re-wiring** them.
- Programs written in machine or assembly language.
- Problems:
  - Serial processing: users had access to the computer one by one in series.
  - Users had to write the same routines again and again.



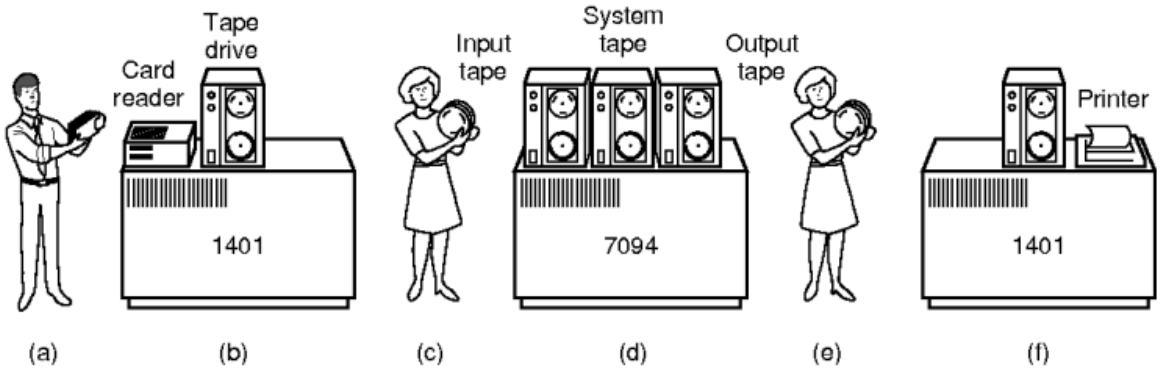
http://ysfine.com/wigner/neumann.html

2nd Generation: 1955-1965

- Separation between operators and programmers.
  - The programmer prepares her/his job offline.
  - The operator runs the job and delivers a printed output.
- Programs are in FORTRAN or assembly language
  - A programmer would punch it on cards.
- Batch the jobs together.
  - The operator pre-reads jobs onto a magnetic tape.
  - The operator loads a special program (monitor) that reads the jobs from the tapes and runs them sequentially.
  - The monitor program writes the output of each job on a second magnetic tape.
- Problems:
  - A lot of CPU time was wasted waiting for I/O instructions to complete.
  - I/O devices were much slower than processors (especially tapes!)



Courtesy: http://www.columbia.edu/cu/computinghistory/1965.html

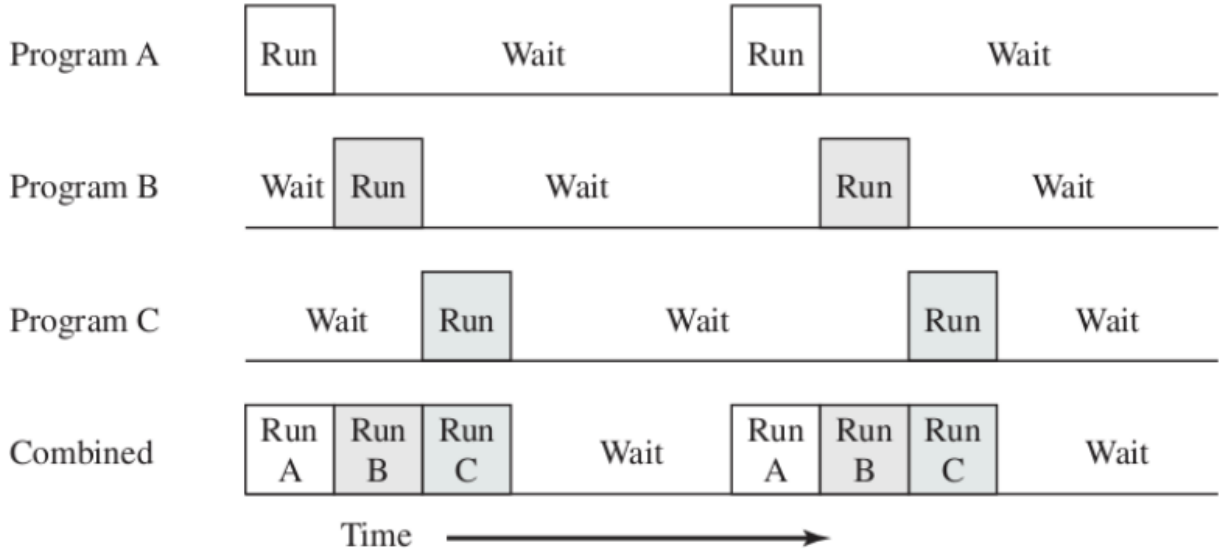


Courtesy: A.S. Tanenbaum et al., Operating Systems Design and Implementation, 2006

3rd Generation: 1965-1980

- **Multiprogrammed batch systems**.
- Jobs are kept in main memory simultaneously, and the CPU is multiplexed or multiprogrammed among them.
- Tasks kept running until they performed an operation that required waiting for an external event, such as I/O.
- Problems: in a multiple-user system, users want to see their program running as if it were the only program on the computer.
- Solution: **time-sharing**, Birth of UNIX!

Courtesy: W. Stallings, Operating Systems: Internals and Design Principles, 2011



UNIX®

4th Generation: 1980-present

- Personal Computers
- Transition from human operators to software (Operating Systems)
  - From multiple users back to a single user.
  - Multitasking is a central feature of modern PC operating systems.
  - PC systems emphasize user convenience
- We mostly focus on Linux in this course

- GNU (GNU's Not Unix!): 1983



- Mac OS: 1984



- Microsoft Windows: 1985

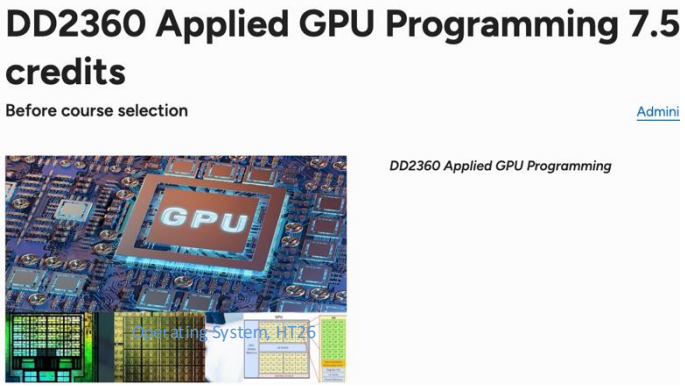


- Linux: 1991



5th Era: 1990s-Present

- The 4th generation gave each user a machine. The 5th decouples the OS from the machine.
- **Virtualization - one machine, many OSes**
  - A **hypervisor** multiplexes hardware among whole operating systems, exactly as an OS multiplexes a CPU among processes.
  - Not new: IBM VM/370, 1972. What changed is **cost**: hardware support (Intel VT-x 2005, AMD-V 2006) made it nearly free.
  - Enables the cloud: Amazon EC2 (2006) rents you a virtual machine, not a computer.
- **Containers - one OS, many isolated worlds**
  - Isolation **inside** one kernel: Linux **namespaces** (what a process sees) + **cgroups** (what it consumes). Docker 2013, Kubernetes 2014.
- **Mobile** - iOS 2007, Android 2008. Android is the most widely used OS with Linux kernel.
- **Accelerators** - GPUs is becoming one most popular accelerator.



Different Kernel Structures

Structure	What runs in the kernel?	Example	Pros	Cons
Monolithic	Everything: scheduling, memory, file systems, network	**Linux**, UNIX	Fast	One bad driver panics the whole machine
Microkernel	Minimum: address spaces, threads, IPC.	Minix3, QNX	Fault isolation	Every service call becomes IPC
Modular	Core kernel + modules loaded at run time	Linux LKMs, Solaris	Extensible without reboot	no isolation gained
Hybrid	Microkernel core wrapped in monolithic layers	macOS, Windows NT	-	-

Linux is monolithic and modular

Demo: Run 'lsmod | head -n 20'

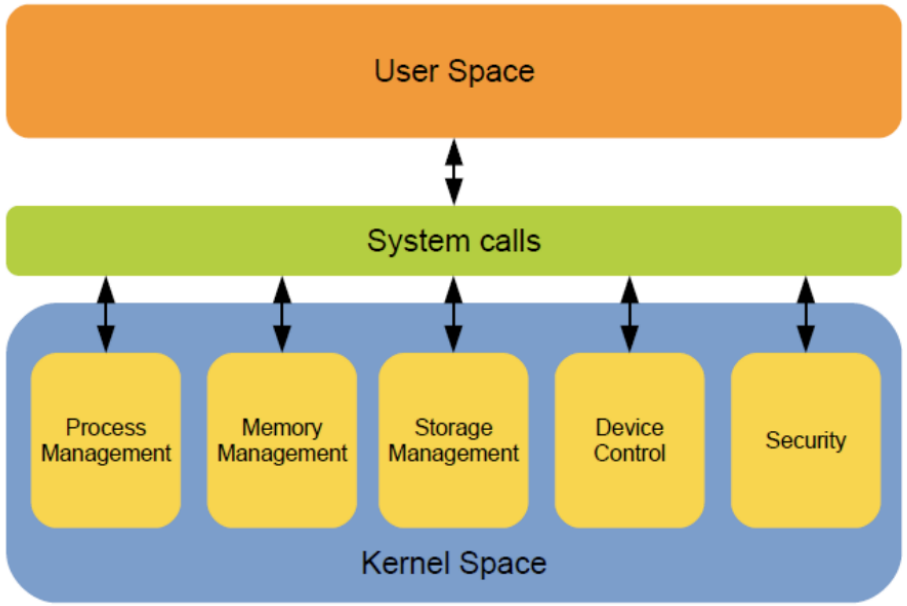


Outline

- What is an OS
- A brief history (video lecture)
- **OS organization**
- OS structure

OS Organization

- Basic organization of an Operating System
  - Process management
  - Memory management
  - Storage management and File system
  - Device control and I/O subsystem
  - Protection and security



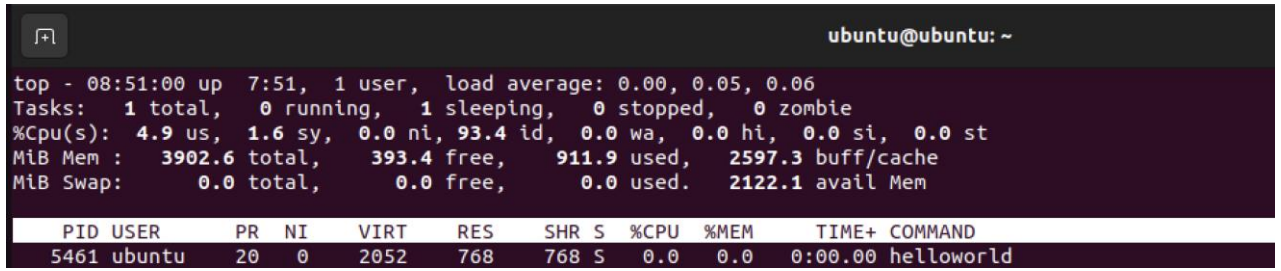
OS Organization: Process Management

- A **process is a program in execution**.
  - The program is a passive entity, and the process is an active entity.
- A process needs resources to accomplish its task.
  - CPU, memory, I/O, files, initialization data, ...
- Process termination requires the reclaim of any reusable resources.

```
#include <stdio.h>

void main() {
  printf("Hello World!\n");
}
```

program



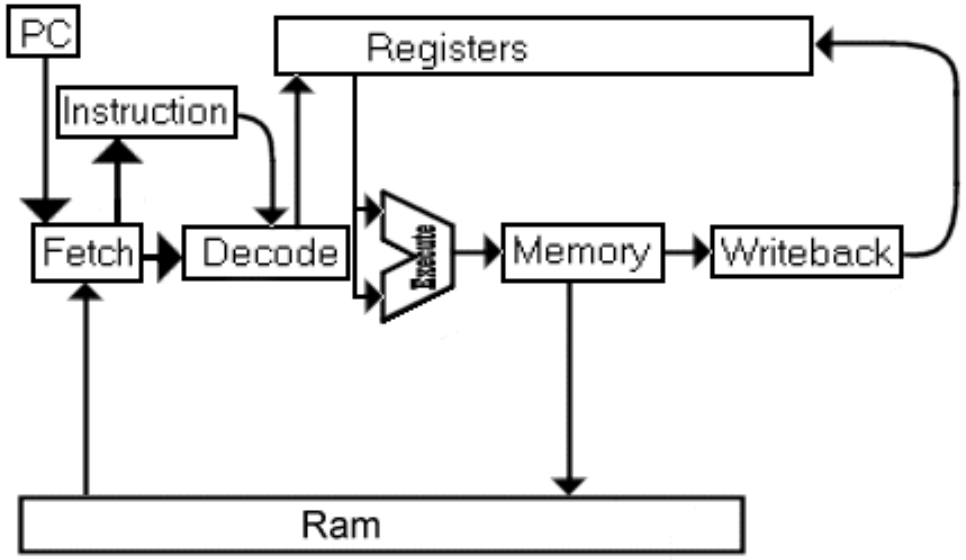
process

OS Organization: Process Management

- Main activities:
  - Creating and deleting both user and system processes.
  - Scheduling processes and threads on the CPUs.
  - Suspending and resuming processes.
  - Process synchronization.
  - Inter-process communication.

OS Organization: Memory Management

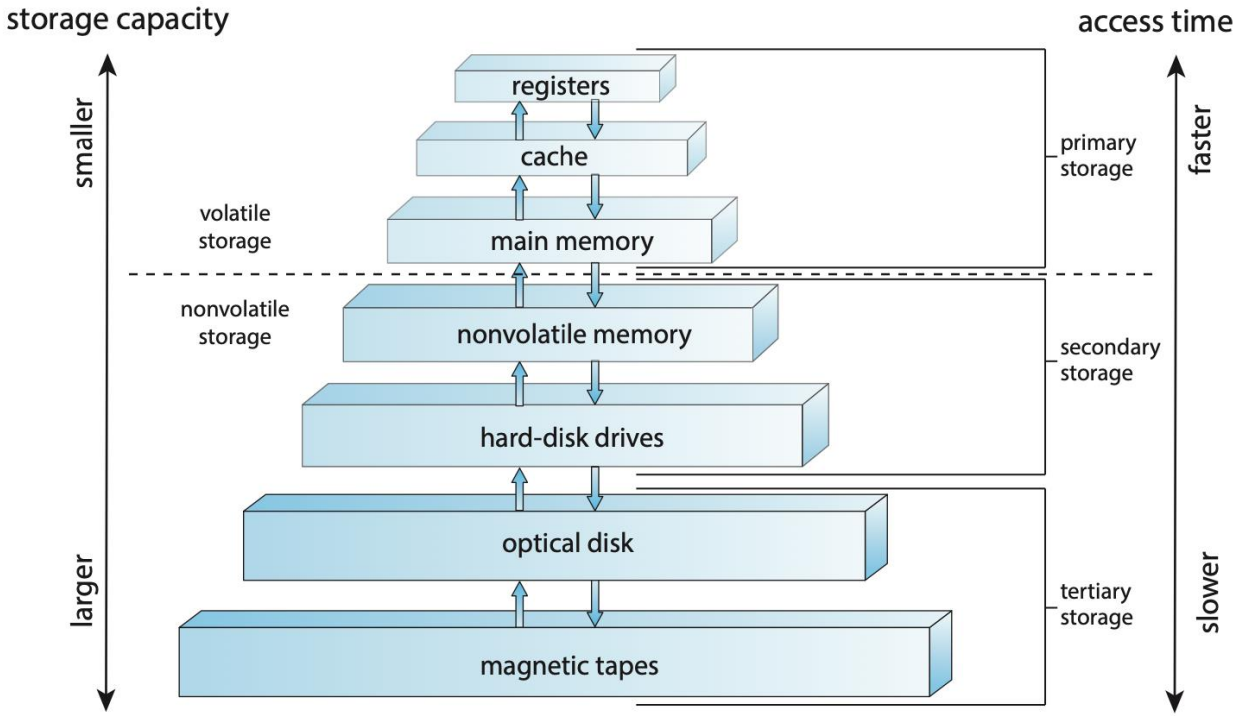
- To execute a program, all (or part) of the instructions must be in memory.
  - Recall that an instruction has 5 stages of execution: Fetch, Decode, Execute, Memory, and Writeback
- All (or part) of the data needed by the program must be in memory.



A simple 5-stage instruction execution pipeline

OS Organization: Memory Management

- Goal: optimize CPU utilization and computer response by placing data and instructions in main memory
- Main activities:
  - Track which parts of memory are currently being used and by whom.
  - Decide which processes and data to move into and out of memory.
  - Allocate and Deallocate memory space as needed.



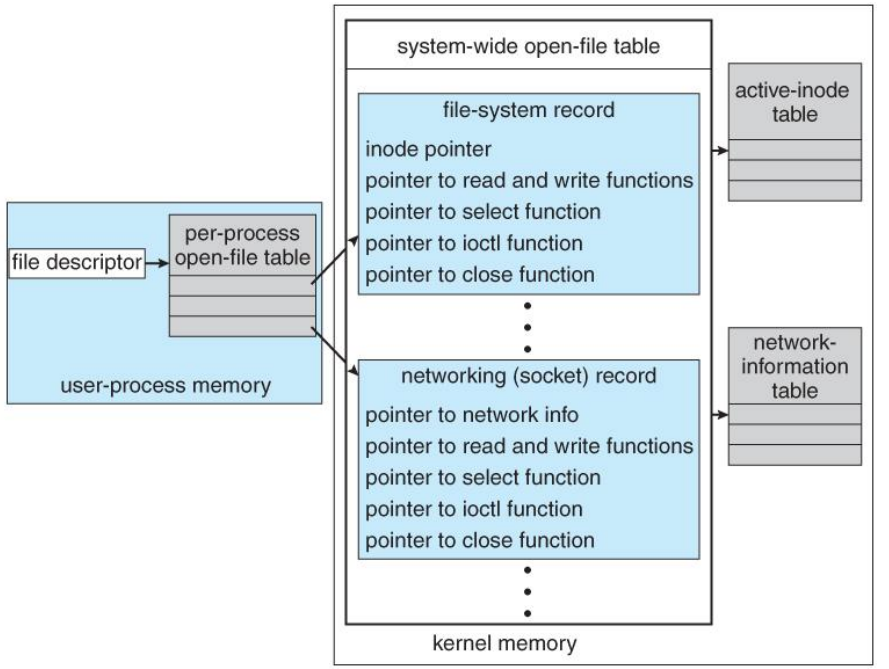
OS Organization: Storage Management

- Disks are usually used to store data that does not fit in main memory or data that must be kept for a long period of time.
- Disk management activities:
  - Free-space management
  - Storage allocation
  - Disk scheduling



OS Organization: Storage Management

- OS provides a uniform and logical view of information storage.
- OS abstracts physical properties to the logical storage unit called a file.
  - A file is a collection of related information (programs or data).
  - Files are usually organized into directories.
- OS maps files onto physical media and accesses these files via the storage devices, e.g., disk drive, tape drive.

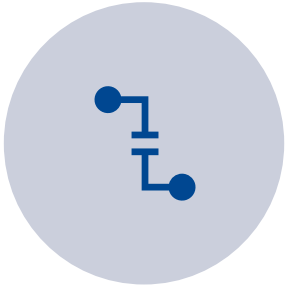


The kernel maintains a data structure “open file table”

OS Organization: File Systems



Creating and deleting files and directories.



Primitives to manipulate files and directories.



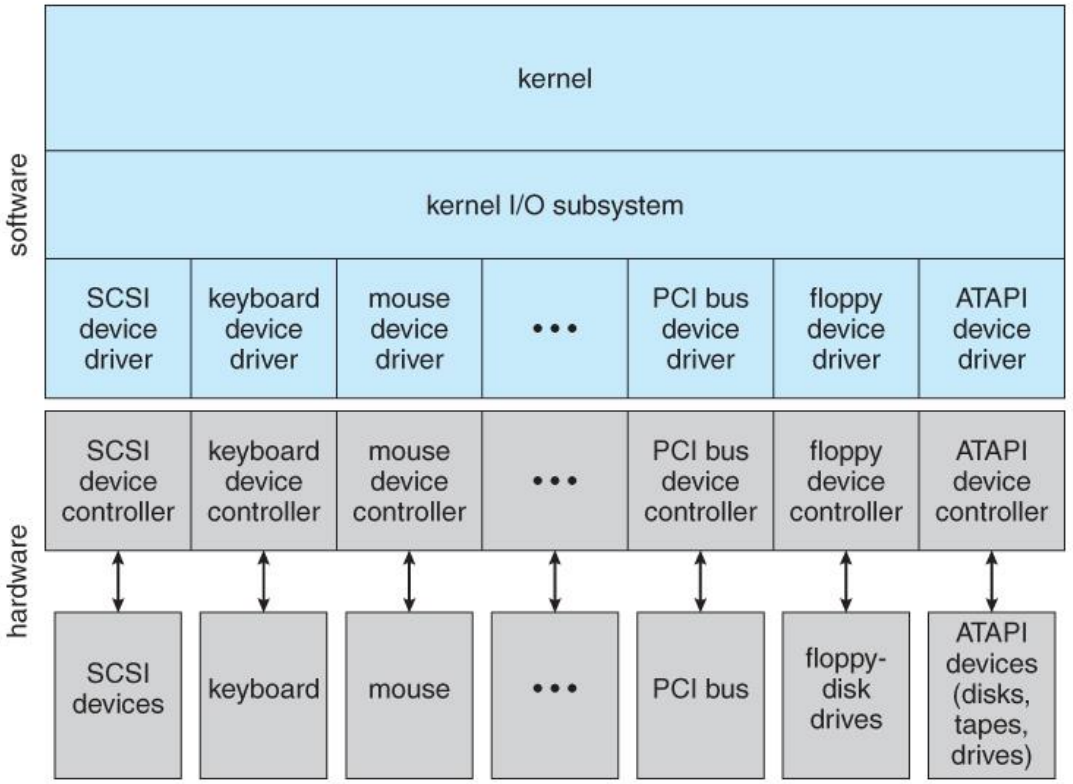
Mapping files onto secondary storage.



Backup files onto stable (non-volatile) storage media.

OS Organization: Device Control and I/O subsystem

- OS manages a list of device drivers, one for each device
- Device drivers is a software:
  - provide a uniform interface between OS and Hardware
  - Translate OS operations (e.g., read()) into commands that can be understood by device controllers
- A device controller located on the hardware (e.g., printer):
  - Receive commands from the driver
  - Control/communicate in the hardware device
  - Send interrupts to CPU



OS Organization: Device Control and I/O subsystem

- Main I/O subsystem in the Kernel:
  - I/O Scheduling: e.g., scheduling of disk accesses
  - Buffering and caching: keep a copy of data in a faster tier to bridge speed differences between two devices
  - Error Handling, e.g., buffer overflow, disk crash, memory error (OS bookkeeping faulty memory pages)
  - I/O Protection: all I/O requests are handled through system calls, memory mapped areas and I/O ports are protected by the memory management system

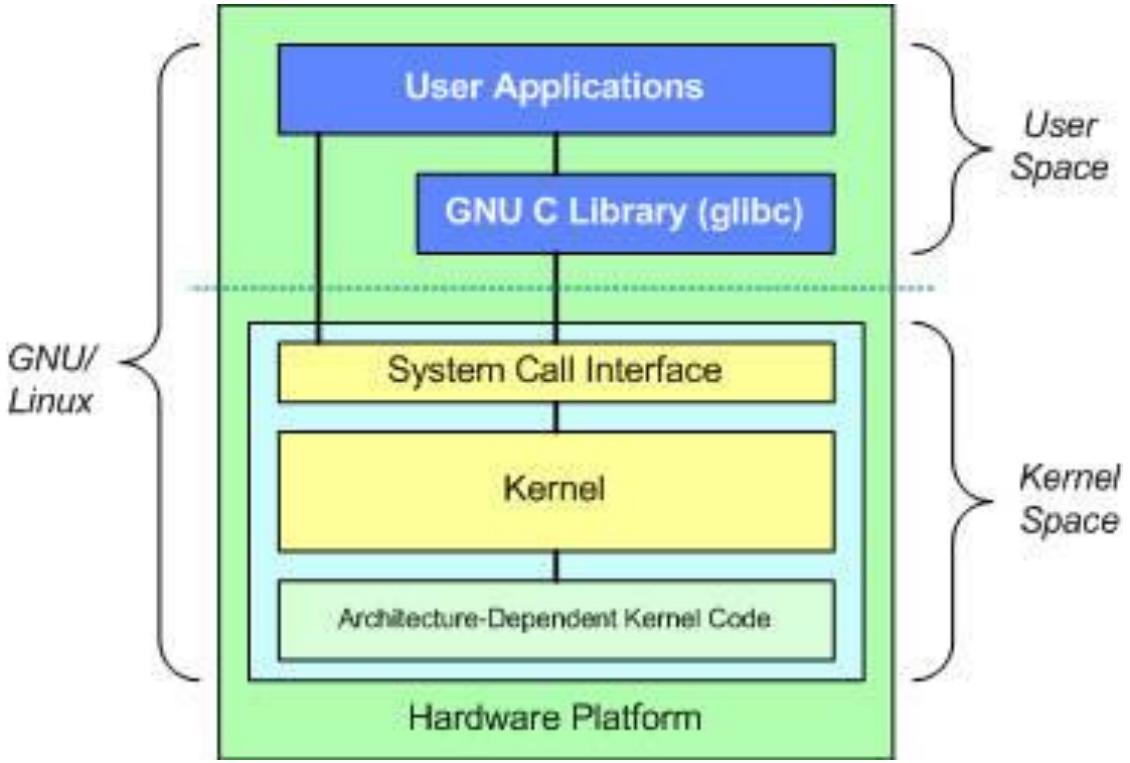


Outline

- What is an OS
- A brief history (video lecture)
- OS organization
- **OS structure**

Operating System Structure

- The operating system is the one program always running on the computer, usually called the **kernel**
  - Linux kernel is monolithic
- Everything else is either:
  - a **system program**
  - an **application program**.



Courtesy: https://developer.ibm.com/articles/l-linux-kernel/

System Programs

- An environment for program development and execution.
- System programs include:
  - File manipulation, e.g., copy, delete, rename, and edit files
  - Status information, e.g., date, time, and available memory
  - Programming language support, e.g., assemblers and debuggers
  - Program loading and execution, e.g., loaders
  - Communications, e.g., services to make connections among processes, users, and hardware
  - Background services, e.g., services and daemons

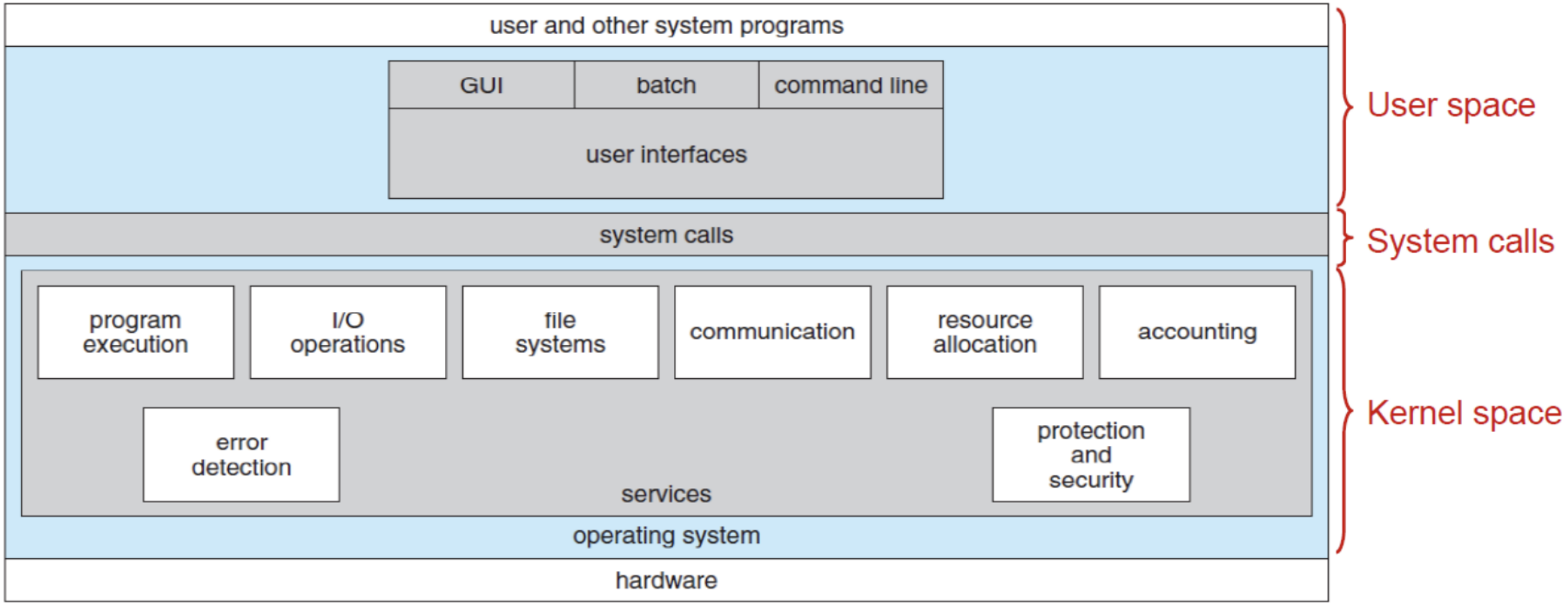
Application Programs

- Don’t pertain to the system.
- Run by users.
- Not typically considered part of OS.
- Launched by command line, mouse click, finger poke.
- Web browsers, word processors, database systems, compilers, games, ...



System Calls

- System calls provide system services in kernel space to user space
  - Typically written in a high-level language (C or C++).



Types of System Calls

- System calls can be grouped roughly into six major categories:
  - Process control
  - File manipulation
  - Device manipulation
  - Information Maintenance
  - Communications
  - Protection

- In our assignments, we will practice using Unix system calls.

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

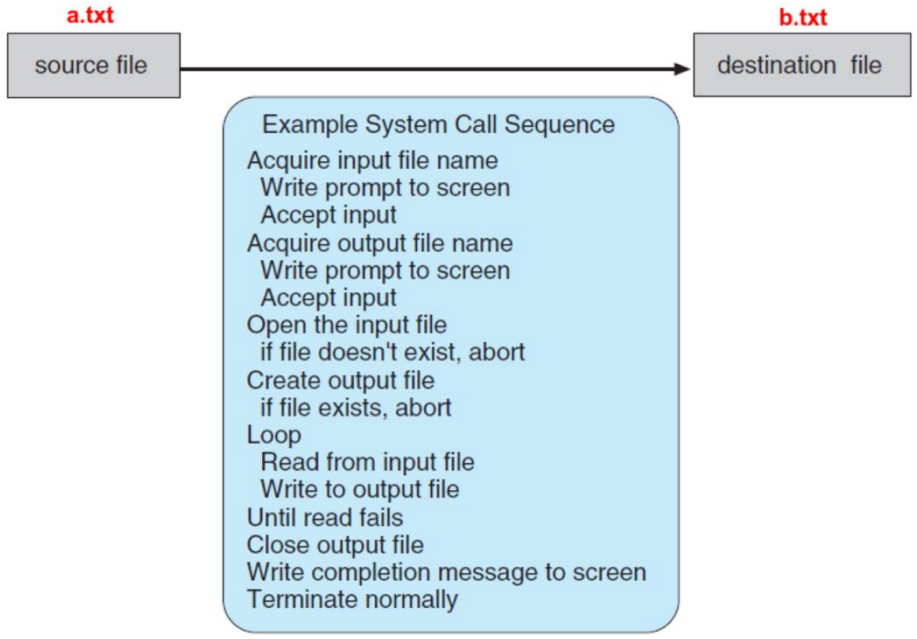
	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

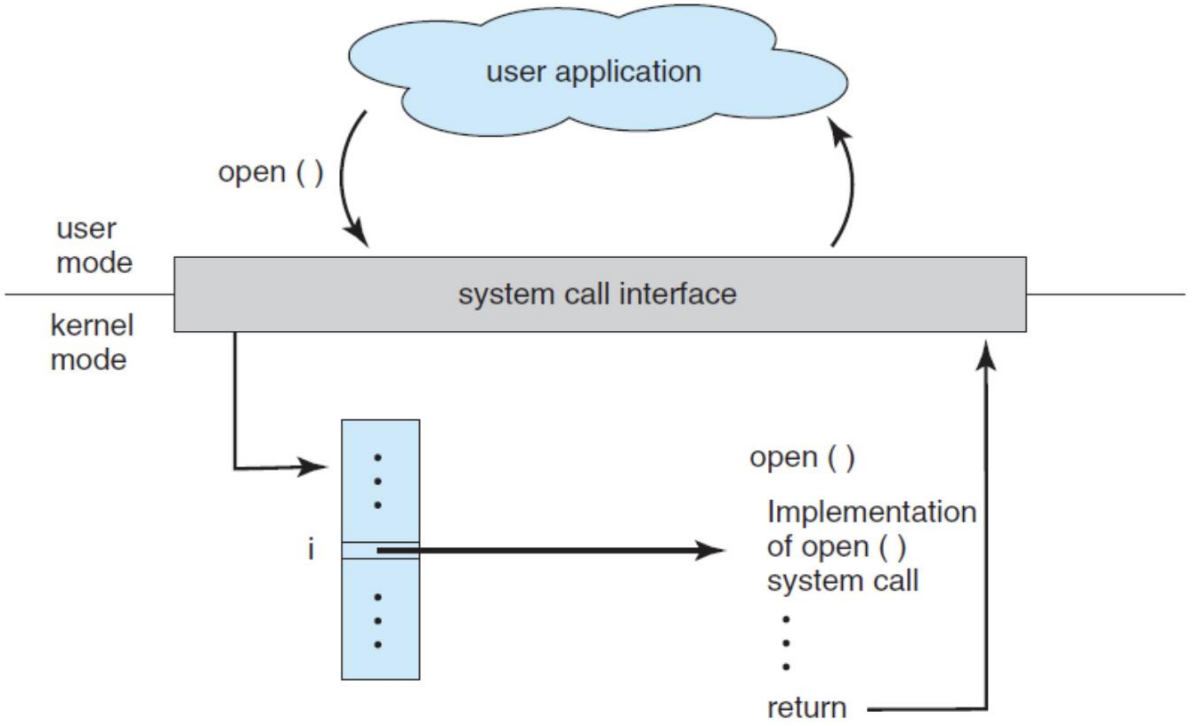
Application Programming Interface (API)

- The API specifies a set of functions that are available to an application programmer.
  - It includes the parameters that are passed to each function and the return values the programmer can expect.
- The three most common APIs:
  - POSIX API for POSIX-based systems (including all versions of UNIX, Linux, and Mac OS X)
  - Windows API for Windows
  - Java API for the Java virtual machine (JVM)

API and System Calls

```
> cp a.txt b.txt
```



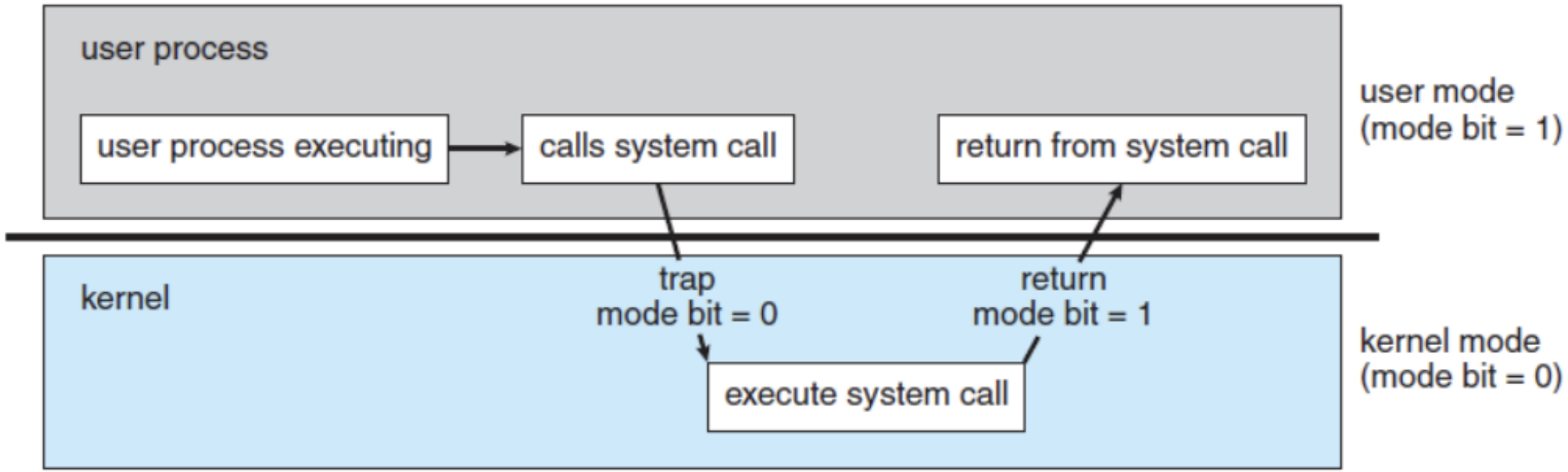


API and System Calls: strace

```
> strace cp a.txt b.txt
execve("/bin/cp", ["cp", "a.txt", "b.txt"], [/* 49 vars */]) = 0
brk(0)                                  = 0x8a2d000
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
mmap2(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb76ff000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
open("/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=108563, ...}) = 0
mmap2(NULL, 108563, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb76e4000
close(3)                                = 0
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/lib/i386-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\1\1\1\0\0\0\0\0\0\0\0\3\0\3\0\1\0\0\0@A\0\0004\0\0\0"..., 512) = 512
fstat64(3, {st_mode=S_IFREG|0644, st_size=120748, ...}) = 0
mmap2(NULL, 125852, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0xb76c5000
mmap2(0xb76e2000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1c) = 0xb76e2000
close(3)                                = 0
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/lib/i386-linux-gnu/librt.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\1\1\1\0\0\0\0\0\0\0\0\3\0\3\0\1\0\0\0\320\30\0\0004\0\0\0"..., 512) = 512
fstat64(3, {st_mode=S_IFREG|0644, st_size=30684, ...}) = 0
mmap2(NULL, 33360, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0xb76bc000
mmap2(0xb76c3000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x6) = 0xb76c3000
close(3)                                = 0
...
```

Dual-Mode Operation

- A hardware mechanism to let an OS protect itself and other system components
- CPU runs in either User mode or Kernel mode
  - mode bit, privilege instructions
  - The system call changes mode to the kernel, and the return from the call resets it to the user.



The transition from user to kernel mode

Interrupt, Trap, Fault

A user program holds the CPU. How does the kernel ever get the CPU back?

	Interrupt	Trap	Fault
Example	Disk finished; timer fired; key pressed	The syscall instruction	Page fault; divide-by-zero error
Timing	Asynchronous	Synchronous	Synchronous

A hardware path: event occurs (e.g., I/O event finishes) → CPU finishes the current instruction → index the interrupt vector table → switch to kernel mode → run the registered handler → restore state, return to user mode.

Example: the timer interrupt

Summary

- OS is a layer between application programs and hardware
- OS operations
- System calls
- Multiprogramming
- Dual mode

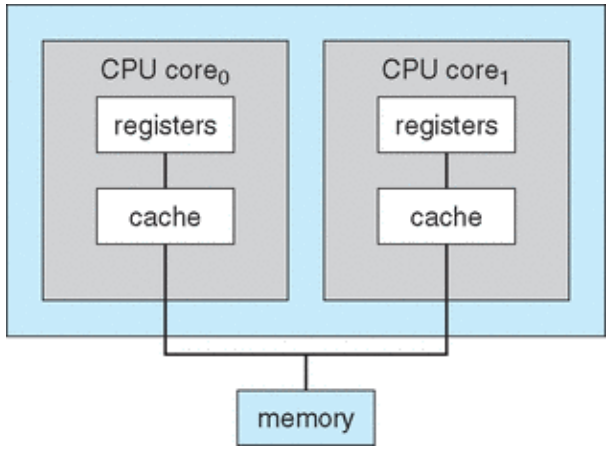
Optional: Common Data Structures in Kernel

- Lists, Stacks, and Queues
  - singly linked list, doubly linked list, circularly linked list
  - Stack: last in, first out (LIFO) for adding and removing items
  - Queue: first in, first out (FIFO) for adding and removing items
- A tree data structure represents data hierarchically
  - parent–child relationships
  - logarithmic complexity in balanced tree structures
- Hash Functions and Maps
  - hash collision
- A bitmap uses n binary digits to represent the status of n items.

Optional: Computer System Architectures

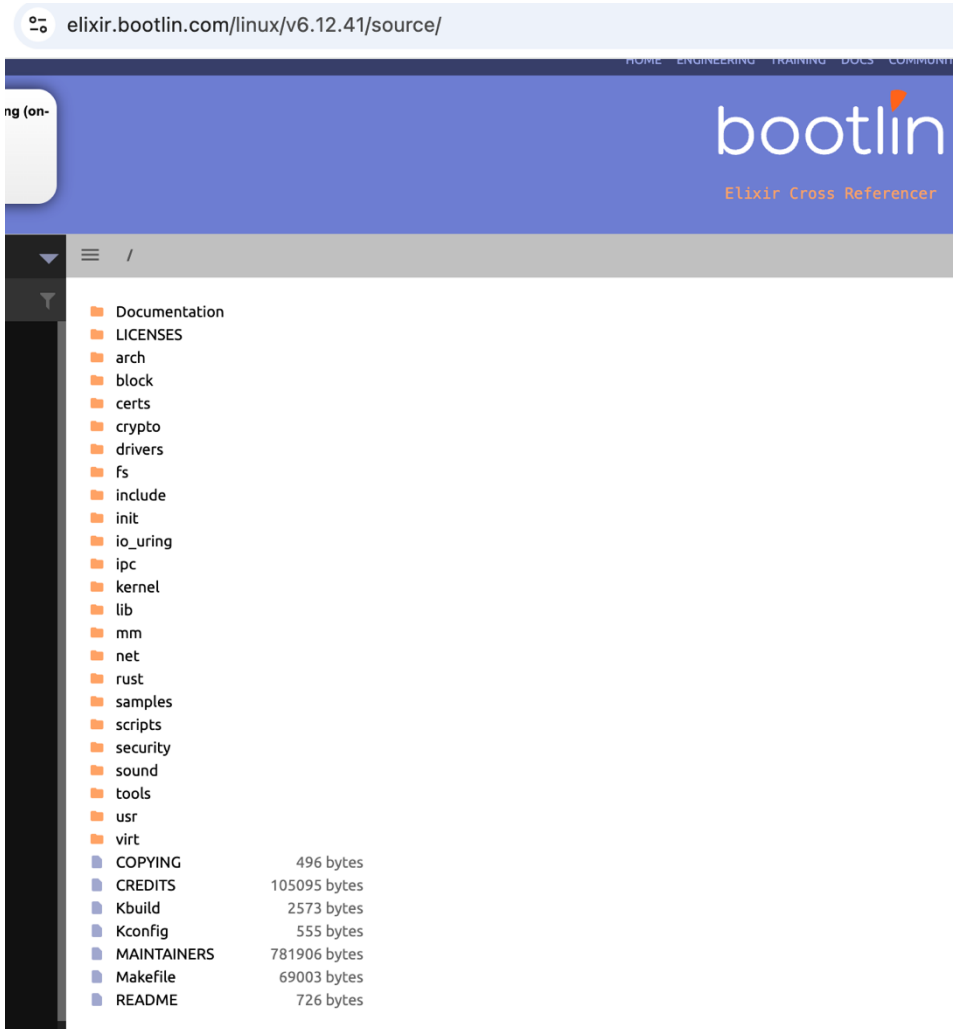
- Single Processor Systems: one main CPU
  - Most course content is based on this scenario
- Multiprocessor Systems: multi CPUs/cores
  - Symmetric multiprocessing systems (SMP)
  - NUMA: non-uniform memory access
- Clustered Systems:
  - HPC: high-performance computing
  - Two or more nodes interconnected by network

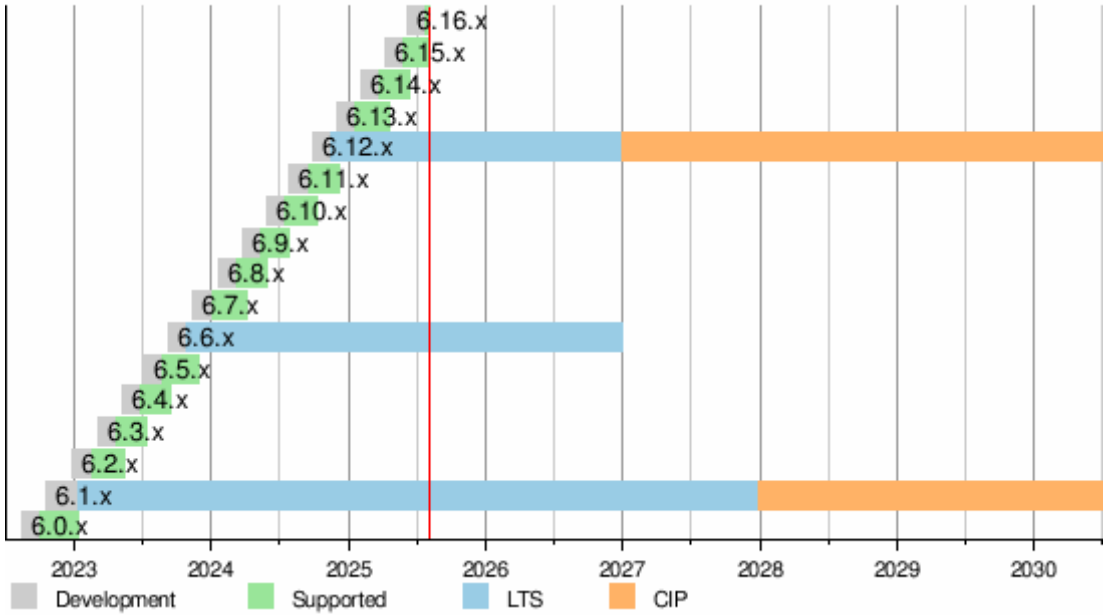
An SMP system



Optional: Linux Implementation

- If you are curious of Linux's implementation, try to check a Long-term support Linux version, e.g., https://elixir.bootlin.com/linux/v6.12.41/source/





https://en.wikipedia.org/wiki/Linux_kernel_version_history#Releases_6.x.y

Next Step

- Watch the video clips on processes before Nov 03 lecture

